

面向对象程序设计 (C++) 大题专项训练

—— 参 考 答 案 ——

说明：① 程序阅读题（专项三）的答案均为程序在 g++ 编译器下实际编译运行得到的输出；② 类的设计题（专项二）只要求给出类的框架及类之间的关系，不要求完整实现；③ 综合编程题（专项五）的答案为考试作答风格的框架式写法，给出类体系、关键的多态接口和 main 调用过程即可，具体数据和细节可在考试时任意设置。

专项一 程序改错题

1. (2 分)

tick() 被声明为 const 成员函数，但函数体中修改了数据成员 m。const 成员函数承诺不修改对象的状态（其 this 指针为指向常量的指针），因此 m++ 编译报错。改正：去掉 tick() 后面的 const（本题 tick 的语义就是要修改对象，不宜改用 mutable）。

2. (2 分)

类中已经定义了有参构造函数 Point(int,int)，编译器不再自动生成默认（无参）构造函数，因此 Point p2; 找不到可匹配的构造函数，编译错误。改正：补充一个无参构造函数，如 Point(){ x=0; y=0; }; 或者给原构造函数的参数加默认值：Point(int a=0, int b=0)。

3. (4 分，两处错误)

错误一：拷贝构造函数的参数必须是（常）引用。写成按值传递 Buffer(Buffer b) 时，初始化形参 b 又需要调用拷贝构造函数，造成无穷递归，C++ 规定这是非法的，编译错误。应改为 Buffer(const Buffer& b)。

错误二：data 是用 new[] 申请的数组，析构时必须用 delete[] data; 释放，而写成 delete data; 行为未定义。

此外不足：拷贝构造采用浅拷贝，b1 与 b2 的 data 指向同一块内存，两个对象析构时同一块内存被释放两次。应改为深拷贝（申请新空间并复制内容），同时类中还应相应地重载赋值运算符。

4. (2 分)

基类 Animal 的析构函数不是虚函数。delete p 时只按指针的静态类型调用 ~Animal()，派生类 Dog 的析构函数不会被执行，name 指向的内存泄漏。改正：把基类析构函数声明为虚函数 virtual ~Animal()，这样通过基类指针删除派生类对象时会先调用 ~Dog() 再调用 ~Animal()，资源被正确释放。

5. (2 分)

report() 是静态成员函数，没有 this 指针，不能直接访问非静态数据成员 balance（balance 属于某个具体对象，而静态函数不与任何对象关联），编译错误。改正：report() 中只输出静态成员 count；若确实需要输出余额，可把 report() 改为普通成员函数，或把对象（引用）作为参数传给 report()。

专项二 类的设计题

(以下各题给出类的框架及类之间的关系即可, 成员细节用 // ... 省略)

1. 智能家居系统

```
class Device {                // 抽象设备基类
public:
    virtual void on() = 0;
    virtual void off() = 0;
    // ...
};
class Light : public Device {
    // 智能灯, 可调亮度
};
class AirCond : public Device {
    // 空调, 可调温度
};
// 以下省略其余设备类 Curtain、Speaker 等, 写法同上
class Controller {           // 抽象控制方式基类
public:
    virtual void control(Device* d /*, 具体操作 */ ) = 0;
    // ...
};
class AppController : public Controller {
    // 手机App控制
};
class VoiceController : public Controller {
    // 语音控制
};
// 以下省略定时控制 TimerController、传感器联动 SensorController
class Scene {                 // 场景: 一组设备及其动作的编排
    string name;
    vector<Device*> devices;
public:
    void addDevice(Device* d /*, 动作 */);
    void run();                // 一键执行场景
};
class SmartHome {             // 智能家居总控
    vector<Device*> devices;
    vector<Controller*> controllers;
    vector<Scene*> scenes;
public:
    void addDevice(Device* d);
    void addScene(Scene* s);
    void runScene(string name);
    // ...
};
```

新增设备、控制方式或场景时只需增加新的派生类或新的 Scene 对象, 无需修改已有代码, 满足可复用、可扩展、可维护。

2. 饮品店点单计价

```
class Drink {                                // 抽象饮品基类
protected:
    float basePrice;
public:
    virtual float price() = 0; // 计算饮品本身的价格
    // ...
};
class Coffee : public Drink {
    // ...
};
class MilkTea : public Drink {
    // ...
};
// 以下省略果茶 FruitTea 等饮品类
class Spec {                                // 规格：杯型、冷热等，影响价格
    // 大中小杯加价、冷热 ...
public:
    float extra();
};
class Topping {                             // 抽象小料基类
public:
    virtual float price() = 0;
};
class Pearl : public Topping { /* 珍珠 */ };
class Cream : public Topping { /* 奶盖 */ };
// 以下省略椰果 Coconut 等小料类
class Discount {                            // 抽象优惠方式基类
public:
    virtual float apply(float total) = 0; // 传入原价返回优惠后价格
};
class VipDiscount : public Discount { /* 会员折扣 */ };
class SecondHalf : public Discount { /* 第二杯半价 */ };
// 以下省略满减 FullCut 等优惠类
class Order {                               // 一笔订单
    vector<Drink*> drinks; // 每杯饮品带各自的规格与小料
    Discount* discount;
public:
    void addDrink(Drink* d);
    float total(); // 汇总各杯价格并应用优惠
};
class Shop {
    vector<Order*> orders;
    // ...
};
```

3. 快递寄运系统

```
class Goods {                                // 抽象物品基类
protected:
    float weight, volume;
public:
    virtual float packFee() = 0;    // 不同物品的包装/附加要求
    // ...
};
class Document : public Goods { /* 普通文件 */ };
class Fragile : public Goods { /* 易碎品 */ };
// 以下省略生鲜冷链 FreshGoods、大件 BigGoods
class Transport {                          // 抽象运输方式基类
public:
    virtual float fee(Goods* g, float distance) = 0;
};
class AirTransport : public Transport { /* 航空 */ };
class RoadTransport : public Transport { /* 公路 */ };
// 以下省略铁路 RailTransport 等
class PriceRule {                          // 抽象计费规则（重量/体积/保价等）
public:
    virtual float charge(Goods* g) = 0;
};
class WeightRule : public PriceRule { /* 按重量计费 */ };
class InsureRule : public PriceRule { /* 保价费 */ };
// 以下省略按体积计费 VolumeRule 等
class Express {                            // 快递公司：组合运输方式与资费规则
    string name;
    vector<Transport*> ts;
    vector<PriceRule*> rules;
public:
    float charge(Goods* g, float distance /*, 选择的运输方式 */);
};
class Order {                              // 一笔寄运订单
    Goods* g;
    Express* company;
    // 寄件人、收件人 ...
public:
    float total();
};
```

4. 医院门诊预约挂号

```
class Doctor { // 抽象医生基类（按职称派生）
protected:
    string name;
public:
    virtual float regFee() = 0; // 挂号费随职称不同
};
class ChiefDoctor : public Doctor { /* 主任医师 */ };
class DeputyDoctor : public Doctor { /* 副主任医师 */ };
// 以下省略主治医师 AttendingDoctor 等
class Department { // 科室
    string name;
    vector<Doctor*> doctors;
};
class TicketType { // 抽象号源类型
public:
    virtual float price(Doctor* d) = 0;
};
class NormalTicket : public TicketType { /* 普通号 */ };
class ExpertTicket : public TicketType { /* 专家号 */ };
// 以下省略急诊号 EmergencyTicket
class Channel { // 抽象预约渠道
public:
    virtual Appointment* book(/* 科室、医生、号源 */) = 0;
};
class WindowChannel : public Channel { /* 现场窗口 */ };
class AppChannel : public Channel { /* 手机App */ };
// 以下省略电话渠道 PhoneChannel
class Payment { // 抽象支付方式
public:
    virtual void pay(float fee) = 0;
};
class InsurancePay : public Payment { /* 医保支付 */ };
class SelfPay : public Payment { /* 自费支付 */ };
class Appointment { // 一次预约记录
    Doctor* doctor;
    TicketType* type;
    Payment* payment;
    // 患者、就诊时间 ...
};
class Hospital { // 医院：统一对外提供预约服务
    vector<Department*> deps;
    vector<Channel*> channels;
    // ...
};
```

5. 自动售货机

```
class Product {                // 商品（也可按饮料/零食/文具再派生）
    string name;
    float price;
    // ...
};
class Payment {                // 抽象支付方式基类
public:
    virtual bool pay(float money) = 0;
};
class CoinPay : public Payment { /* 投币 */ };
class QrPay : public Payment { /* 扫码 */ };
// 以下省略刷脸支付 FacePay
class Promotion {             // 抽象促销策略基类
public:
    virtual float apply(float total) = 0;
};
class SecondHalf : public Promotion { /* 第二件半价 */ };
class FullCut : public Promotion { /* 满10元减2元 */ };
// ...
class Inventory {             // 库存管理
    map<Product*, int> stock;
public:
    bool out(Product* p);      // 出货，库存不足返回false并提示补货
    void replenish(Product* p, int n); // 补货
};
class VendingMachine {        // 售货机：组合以上各部分
    Inventory inv;
    vector<Payment*> pays;
    vector<Promotion*> promos;
public:
    void sell(/* 商品、数量、支付方式 */); // 计价→促销→支付→出货
};
```

专项三 程序阅读题

(以下输出均为 g++ 实际编译运行的结果)

1. (10 分) 程序输出:

```
T0 in
-- main --
T3 in
-- f --
T1 in
T2 in
T1 out
-- f --
T1 in
T1 out
-- end --
T3 out
T2 out
T0 out
```

【说明】全局对象 g 在 main 之前构造、程序结束时最后析构；T x(); 是函数声明而不是对象定义，不产生任何输出；函数内局部对象每次调用都构造和析构，static 局部对象只在第一次调用时构造一次，程序结束时与全局对象一起按构造的逆序析构（先 T2 后 T0）。

2. (10 分) 程序输出:

```
Base::play
Base::step
Base::jump(double)
Base::play
Derived::step
Derived::jump(int)
Base::jump(double)
```

【说明】Derived 中的 jump(int) 与基类 jump(double) 参数不同，属于名字隐藏而不是覆盖：Derived::step 中调用 jump(2) 直接绑定 Derived::jump(int)；而 p->jump(3.5) 按静态类型 Base 进行名字查找，找到的虚函数 jump(double) 未被覆盖，最终执行 Base::jump(double)。play 虽未被覆盖，但其中对 step() 的调用仍按对象的动态类型分派。

3. (10 分) 程序输出:

```
P(1)
copy 11
copy 11
~P 12
assign 102
2,102
~P 102
~P 2
```

【说明】P b = a 与按值传参 byVal(a) 都调用拷贝构造函数（本题中 +10），按引用传参不发生拷贝；函数形参在函数返回时析构（~P 12）；b = a 调用赋值运算符（+100）；main 结束时局部对象按定义的逆序析构（先 b 后 a）。

4. (10 分) 程序输出:

```
[5]
[6]
[6]
[7]
[13],[8]
[5]
```

【说明】前置 ++ 修改对象并返回自身引用；后置 ++ 先保存旧值副本再加 1，返回旧副本（所以 s++ 输出 [6]，之后 s 为 [7]）；s+3 生成新对象不改变 s；Score u = 3 借助转换构造函数把 3 转换为 Score；++(++u) 因前置 ++ 返回引用而可以连续作用，输出 [5]。

5. (10 分) 程序输出:

```
Base()
Base::hello
Part:one
Part:two
Derived()
Derived::hello
Derived::hello
~Derived
~Base
```

【说明】构造顺序为：先基类，再按成员的"声明顺序"（而不是初始化列表中的顺序）构造 p1、p2，最后执行构造函数体；在基类构造函数中调用虚函数不发生多态，执行的是 Base::hello；析构函数为虚函数，通过基类指针 delete 时先 ~Derived 再 ~Base。

专项四 类的补充设计题

(以下给出完整的类定义, 需要补充的即 public: 之后的部分与类外的静态成员定义。各题答案均已通过 g++ 编译运行验证。)

1. 银行账户类 Account

```
class Account {
    char* pOwner;
    int iNo;
    double dBalance;
    static int iNext;           // 用于自动产生唯一账号
public:
    Account(const char* name) {
        pOwner = new char[strlen(name) + 1]; // 姓名深拷贝
        strcpy(pOwner, name);
        iNo = iNext++;           // 自动编号保证唯一
        dBalance = 0;
    }
    Account(const Account& a) { // 拷贝构造: 姓名深拷贝, 账号重新生成
        pOwner = new char[strlen(a.pOwner) + 1];
        strcpy(pOwner, a.pOwner);
        iNo = iNext++;
        dBalance = a.dBalance;
    }
    Account& operator=(const Account& a) { // 赋值: 保留自己的账号
        if (this == &a) return *this; // 防止自赋值
        delete[] pOwner;
        pOwner = new char[strlen(a.pOwner) + 1];
        strcpy(pOwner, a.pOwner);
        dBalance = a.dBalance;
        return *this;
    }
    ~Account() { delete[] pOwner; }
    Account& operator+(double money) { // 存款
        dBalance += money;
        return *this;
    }
    Account& operator-(double money) { // 取款 (考试时可不检查余额)
        dBalance -= money;
        return *this;
    }
    bool operator<(const Account& a) const { // 比较余额
        return dBalance < a.dBalance;
    }
    friend ostream& operator<<(ostream& os, const Account& a) {
        os << "账号:" << a.iNo << " 户主:" << a.pOwner
            << " 余额:" << a.dBalance;
        return os;
    }
};
int Account::iNext = 10001; // 静态成员类外定义并初始化
```

【说明】要点: ① 姓名用 new[] 深拷贝、析构用 delete[]; ② 拷贝构造产生新账号, 赋值运算保留自身账号且必须防自赋值 (main 中 $a2 = a2 + 500$ 正是自赋值); ③ operator+ / operator- 修改对象后返回自身引用; ④ operator<< 必须为友元函数。

2. 运动手环类 Band

```
class Band {
    char* pUser;
    int iSn;
    int iSteps;
    static int iNext;
public:
    Band(const char* name) {
        pUser = new char[strlen(name) + 1];
        strcpy(pUser, name);
        iSn = iNext++;
        iSteps = 0;
    }
    Band(const Band& b) {          // 拷贝构造：序列号重新生成
        pUser = new char[strlen(b.pUser) + 1];
        strcpy(pUser, b.pUser);
        iSn = iNext++;
        iSteps = b.iSteps;
    }
    Band& operator=(const Band& b) {
        if (this == &b) return *this;
        delete[] pUser;
        pUser = new char[strlen(b.pUser) + 1];
        strcpy(pUser, b.pUser);
        iSteps = b.iSteps;        // 序列号保持不变
        return *this;
    }
    ~Band() { delete[] pUser; }
    Band& operator+(int n) { iSteps += n; return *this; }
    Band& operator++() {          // 前置++：先加，返回自身引用
        ++iSteps;
        return *this;
    }
    Band operator++(int) {         // 后置++：先保存旧值副本再加，返回旧副本
        Band old(*this);
        ++iSteps;
        return old;
    }
    friend ostream& operator<<(ostream& os, const Band& b) {
        os << "序列号:" << b.iSn << " 用户:" << b.pUser
            << " 步数:" << b.iSteps;
        return os;
    }
};
int Band::iNext = 1;
```

【说明】要点：前置 ++ 返回 Band&，后置 ++ 用 int 哑元参数区分、按值返回旧副本。注意 cout << b1++ 输出的是增加之前的步数（旧副本由拷贝构造产生，序列号是新生成的）。

3. 会员卡类 VipCard

```
class VipCard {
    char* pHolder;
    int iCardNo;
    int iPoints;
    static int iNext;
public:
    VipCard(const char* name) {
        pHolder = new char[strlen(name) + 1];
        strcpy(pHolder, name);
        iCardNo = iNext++;
        iPoints = 0;
    }
    VipCard(const VipCard& v) {
        pHolder = new char[strlen(v.pHolder) + 1];
        strcpy(pHolder, v.pHolder);
        iCardNo = iNext++; // 卡号重新生成
        iPoints = v.iPoints;
    }
    VipCard& operator=(const VipCard& v) {
        if (this == &v) return *this;
        delete[] pHolder;
        pHolder = new char[strlen(v.pHolder) + 1];
        strcpy(pHolder, v.pHolder);
        iPoints = v.iPoints; // 卡号保持不变
        return *this;
    }
    ~VipCard() { delete[] pHolder; }
    VipCard& operator+(int p) { // 消费获得积分
        iPoints += p;
        return *this;
    }
    bool operator==(const VipCard& v) const { // 积分相同视为同级会员
        return iPoints == v.iPoints;
    }
    friend ostream& operator<<(ostream& os, const VipCard& v) {
        os << "卡号:" << v.iCardNo << " 持卡人:" << v.pHolder
            << " 积分:" << v.iPoints;
        return os;
    }
};
int VipCard::iNext = 88001;
```

【说明】要点：① 持卡人姓名 char* 必须 new[] 深拷贝、析构 delete[]；② 卡号由静态成员 iNext 自动生成：拷贝构造视为“开新卡”取新卡号，赋值运算只复制持卡人与积分、卡号保持不变（先 delete[] 再 new，并防自赋值）；③ operator+ 累加积分后返回自身引用，operator== 按积分比较应声明为 const；④ operator<< 需要访问私有成员，必须为友元函数。

4. 电表类 Meter

```
class Meter {
    char* pAddr;
    int iMid;
    double dReading;
    static int iNext;
public:
    Meter(const char* addr) {
        pAddr = new char[strlen(addr) + 1];
        strcpy(pAddr, addr);
        iMid = iNext++;
        dReading = 0;
    }
    Meter(const Meter& m) {
        pAddr = new char[strlen(m.pAddr) + 1];
        strcpy(pAddr, m.pAddr);
        iMid = iNext++; // 表号重新生成
        dReading = m.dReading;
    }
    Meter& operator=(const Meter& m) {
        if (this == &m) return *this;
        delete[] pAddr;
        pAddr = new char[strlen(m.pAddr) + 1];
        strcpy(pAddr, m.pAddr);
        dReading = m.dReading; // 表号保持不变
        return *this;
    }
    ~Meter() { delete[] pAddr; }
    Meter& operator+=(double d) { dReading += d; return *this; }
    Meter& operator++() { dReading += 1; return *this; }
    operator double() const { // 类型转换运算符：取出当前读数
        return dReading;
    }
    friend ostream& operator<<(ostream& os, const Meter& m) {
        os << "表号:" << m.iMid << " 地址:" << m.pAddr
            << " 读数:" << m.dReading << "度";
        return os;
    }
};
int Meter::iNext = 30001;
```

【说明】要点：(double)m1 通过类型转换运算符 operator double() 实现，该运算符没有返回类型和参数；cout << m1 时友元 operator<< 是精确匹配，优先于"先转 double 再输出"的转换路径，不会产生二义性。

5. 商品类 Goods

```
class Goods {
    char* pTitle;
    int   iGid;
    int   iStock;
    static int iNext;
public:
    Goods(const char* t) {
        pTitle = new char[strlen(t) + 1];
        strcpy(pTitle, t);
        iGid = iNext++;
        iStock = 0;
    }
    Goods(const Goods& g) {
        pTitle = new char[strlen(g.pTitle) + 1];
        strcpy(pTitle, g.pTitle);
        iGid = iNext++;          // 编号重新生成
        iStock = g.iStock;
    }
    Goods& operator=(const Goods& g) {
        if (this == &g) return *this;
        delete[] pTitle;
        pTitle = new char[strlen(g.pTitle) + 1];
        strcpy(pTitle, g.pTitle);
        iStock = g.iStock;      // 编号保持不变
        return *this;
    }
    ~Goods() { delete[] pTitle; }
    Goods& operator-(int n) {    // 售出n件
        iStock -= n;
        return *this;
    }
    bool operator>(const Goods& g) const {    // 比较库存
        return iStock > g.iStock;
    }
    friend istream& operator>>(istream& is, Goods& g) {    // 进货入库
        int n;
        is >> n;
        g.iStock += n;
        return is;
    }
    friend ostream& operator<<(ostream& os, const Goods& g) {
        os << "编号:" << g.iGid << " 商品:" << g.pTitle
            << " 库存:" << g.iStock;
        return os;
    }
};
int Goods::iNext = 501;
```

【说明】要点：operator>> 必须为友元函数，第二个参数为非 const 引用（要修改库存），并返回 istream& 以支持连续输入。

专项五 综合编程题

(以下为考试作答风格的框架式答案：突出类体系、继承与多态接口和 main 的调用过程，具体数据随便编即可，省略部分用注释说明。)

1. 网约车计费系统

```
class CarType {                // 抽象车型：负责一次行程的基础费用
protected:
    float base, perKm, perMin;  // 起步价、每公里、每分钟单价
public:
    CarType(float b, float k, float m):base(b),perKm(k),perMin(m){}
    virtual float fare(float km, float min) {
        return base + km * perKm + min * perMin;
    }
};

class Economy : public CarType {
public:
    Economy():CarType(8, 1.6, 0.3) {}    // 经济型
};

class Comfort : public CarType {
public:
    Comfort():CarType(12, 2.2, 0.5) {}   // 舒适型
};

class Luxury : public CarType {
public:
    Luxury():CarType(20, 3.5, 0.8) {}    // 豪华型，也可重写fare定制
};

class ExtraRule {              // 抽象附加计费规则
public:
    virtual float apply(float fee, int hour) = 0; // 按出发时刻调整费用
};

class PeakRule : public ExtraRule {    // 高峰倍率
public:
    float apply(float fee, int hour) {
        if ((hour>=7 && hour<9) || (hour>=17 && hour<19))
            return fee * 1.5;
        return fee;
    }
};

class NightRule : public ExtraRule {   // 夜间附加费
public:
    float apply(float fee, int hour) {
        if (hour>=23 || hour<5) return fee + 6;
        return fee;
    }
};

class Coupon {                  // 抽象优惠券
public:
    virtual float use(float fee) = 0;
};

class CashCoupon : public Coupon {    // 立减券
    float cash;
public:
    CashCoupon(float c):cash(c){}
    float use(float fee){ return fee>cash ? fee-cash : 0; }
};
```

```

class RateCoupon : public Coupon {           // 折扣券
    float rate;
public:
    RateCoupon(float r):rate(r){}
    float use(float fee){ return fee * rate; }
};

class Platform {                             // 平台：组合车型与附加规则
    map<string, CarType*> cars;
    vector<ExtraRule*> rules;
public:
    void addCar(string name, CarType* c){ cars[name] = c; }
    void addRule(ExtraRule* r){ rules.push_back(r); }
    float charge(string name, float km, float min,
                  int hour, Coupon* cp = NULL) {
        float fee = cars[name]->fare(km, min);    // 多态计算基础费
        for (int i = 0; i < rules.size(); i++)    // 依次应用附加规则
            fee = rules[i]->apply(fee, hour);
        if (cp) fee = cp->use(fee);                // 最后用券
        return fee;
    }
};

int main(){
    Platform p;
    p.addCar("经济型", new Economy);
    p.addCar("舒适型", new Comfort);
    p.addCar("豪华型", new Luxury);
    p.addRule(new PeakRule);
    p.addRule(new NightRule);
    float total = 0;    // 模拟小李一周内的几次行程，数据随便编
    total += p.charge("经济型", 5.2, 18, 8);    // 早高峰上班
    total += p.charge("舒适型", 12.0, 35, 14,
                     new RateCoupon(0.9));    // 午后出行用9折券
    total += p.charge("经济型", 5.2, 20, 18);    // 晚高峰下班
    total += p.charge("豪华型", 30.5, 50, 23,
                     new CashCoupon(10));    // 深夜去机场用立减券
    cout << "小李本周打车共花费:" << total << "元" << endl;
    return 0;
}

```

2. 农场模拟经营游戏

```
class Season { // 抽象季节：影响生长速度
public:
    virtual float speedRate() = 0;
};
class Spring : public Season {
public:
    float speedRate(){ return 1.2; } // 春天长得快
};
class Winter : public Season {
public:
    float speedRate(){ return 0.5; } // 冬天长得慢
};
// 以下省略夏 Summer、秋 Autumn
class Crop { // 抽象作物
protected:
    float grown; // 已生长进度（天）
    int cycle; // 生长周期
    int price; // 成熟后的售价
public:
    Crop(int c, int p):grown(0),cycle(c),price(p){}
    void grow(Season* s){ grown += s->speedRate(); } // 每天生长
    bool ripe(){ return grown >= cycle; }
    virtual int harvest(){ grown = 0; return price; } // 收获卖钱
};
class Wheat : public Crop {
public:
    Wheat():Crop(3, 12) {} // 小麦：周期3天，卖12金币
};
class Strawberry : public Crop {
public:
    Strawberry():Crop(6, 60) {} // 草莓：周期6天，卖60金币
};
class Animal { // 抽象动物
protected:
    int days, gap, price; // 距上次产出天数、产出间隔、产品售价
public:
    Animal(int g, int p):days(0),gap(g),price(p){}
    virtual int produce(){ // 每天调用，到产出日卖出产品
        if (++days >= gap){ days = 0; return price; }
        return 0;
    }
};
class Hen : public Animal {
public:
    Hen():Animal(1, 3) {} // 母鸡每天下蛋，蛋卖3金币
};
class Cow : public Animal {
public:
    Cow():Animal(2, 15) {} // 奶牛两天产一次奶
};
class Player {
    int gold, exp, level;
public:
    Player():gold(100),exp(0),level(1){}
    void gain(int g){ // 获得金币和经验，经验够了升级
```



```

        gold += g; exp += g;
        if (exp >= level * 50){
            exp = 0; level++;
            cout << "升到" << level << "级, 解锁新作物" << endl;
        }
    }
    // 高等级解锁新作物/动物的判断, 略
};

class Farm {
public:
    // 农场: 组合作物、动物、季节、玩家
    vector<Crop*> crops;
    vector<Animal*> animals;
    Season* season;
    Player* owner;

    Farm(Player* p, Season* s):owner(p),season(s){}
    void plant(Crop* c){ crops.push_back(c); } // 种植 (扣种子钱, 略)
    void addAnimal(Animal* a){ animals.push_back(a); }
    void oneDay(){ // 模拟一天
        for (int i = 0; i < crops.size(); i++){
            crops[i]->grow(season);
            if (crops[i]->ripe())
                owner->gain(crops[i]->harvest());
        }
        for (int i = 0; i < animals.size(); i++)
            owner->gain(animals[i]->produce());
    }
};

int main(){
    Player me;
    Farm farm(&me, new Spring); // 春天开局
    farm.plant(new Wheat);
    farm.plant(new Strawberry);
    farm.addAnimal(new Hen);
    farm.addAnimal(new Cow);
    for (int day = 1; day <= 10; day++) // 模拟经营10天
        farm.oneDay();
    return 0;
}

```

3. 停车场收费系统

```
class ChargeRule {           // 抽象计费规则：一类车一套规则
protected:
    float first, per, daily;   // 首小时价、续费价、每日封顶
public:
    ChargeRule(float f, float p, float d):first(f),per(p),daily(d){}
    virtual float charge(int minutes) {
        int hours = ceil(minutes / 60.0);    // 不足一小时按一小时
        float fee = (hours <= 1) ? first : first + (hours-1)*per;
        return fee < daily ? fee : daily;    // 每日封顶
    }
};

class SmallCar : public ChargeRule {
public:
    SmallCar():ChargeRule(6, 4, 40) {}        // 小型车
};

class LargeCar : public ChargeRule {
public:
    LargeCar():ChargeRule(10, 6, 60) {}       // 大型车
};

class GreenCar : public SmallCar {           // 新能源车：小型车基础上打5折
public:
    float charge(int minutes) {
        return SmallCar::charge(minutes) * 0.5;
    }
};

class MonthlyCar : public ChargeRule {       // 月卡车辆免费
public:
    MonthlyCar():ChargeRule(0, 0, 0) {}
    float charge(int minutes){ return 0; }
};

class ParkingLot {           // 停车场：维护规则表并结算
    map<string, ChargeRule*> rules;
    float income;
public:
    ParkingLot():income(0){}
    void addRule(string type, ChargeRule* r){ rules[type] = r; }
    float settle(string type, int minutes) { // 出场结算
        float fee = rules[type]->charge(minutes); // 多态计费
        income += fee;
        cout << type << "停车" << minutes << "分钟, 收费"
              << fee << "元" << endl;
        return fee;
    }
    float total(){ return income; }
};

class AirportLot : public ParkingLot {
    // 机场停车场派生类，可定制自己的计费规则，略
};

int main(){
    ParkingLot lot;
    lot.addRule("小型车", new SmallCar);
    lot.addRule("大型车", new LargeCar);
    lot.addRule("新能源车", new GreenCar);
    lot.addRule("月卡车", new MonthlyCar);
    // 模拟一天内几辆车进出，停车时长随便编
}
```

```
    lot.settle("小型车", 50);  
    lot.settle("小型车", 200);  
    lot.settle("大型车", 740);  
    lot.settle("新能源车", 130);  
    lot.settle("月卡车", 480);  
    cout << "今日停车场总收入:" << lot.total() << "元" << endl;  
    return 0;  
}
```

4. 银行理财收益计算系统

```
int myrand(){                                // 模拟基金涨跌的简易随机数
    static int seed = 7;
    seed = (seed * 13 + 5) % 21; // 0~20
    return seed;
}
class Product {                             // 抽象理财产品
protected:
    float rate;                             // 年利率
public:
    Product(float r):rate(r){}
    virtual float profit(float money, int days) = 0; // 计算收益
};
class Demand : public Product { // 活期：按日计息，可随时支取
public:
    Demand():Product(0.003) {}
    float profit(float money, int days) {
        return money * rate / 365 * days;
    }
};
class Fixed : public Product { // 定期：存满期限按定期利率
protected:
    int term;                             // 期限（天）
public:
    Fixed(float r, int t):Product(r),term(t){}
    float profit(float money, int days) {
        if (days >= term)
            return money * rate / 365 * days;
        Demand d;                         // 提前支取只能按活期计息
        return d.profit(money, days);
    }
};
class LargeCD : public Fixed { // 大额存单：起存20万，利率上浮
public:
    LargeCD(float r, int t):Fixed(r, t){}
    float profit(float money, int days) {
        if (money < 200000) {
            cout << "未达到起存金额" << endl;
            return 0;
        }
        return Fixed::profit(money, days);
    }
};
class Fund : public Product { // 基金：收益随市场波动，可能亏损
public:
    Fund():Product(0) {}
    float profit(float money, int days) {
        float r = (myrand() - 8) / 100.0; // -8%~+12%，随便模拟
        return money * r;
    }
};
class Holding {                             // 一笔持仓
public:
    Product* p;
    float money;
    int days;
```

```

    Holding(Product* pp, float m, int d):p(pp),money(m),days(d){}
};
class Customer {
    vector<Holding*> hs;
public:
    void buy(Product* p, float money, int days) {
        hs.push_back(new Holding(p, money, days));
    }
    float yearProfit() {          // 汇总各笔收益
        float total = 0;
        for (int i = 0; i < hs.size(); i++)
            total += hs[i]->p->profit(hs[i]->money,
                                     hs[i]->days);    // 多态
        return total;
    }
};
int main(){
    Customer wang;                // 模拟王同学的投资组合，数据随便编
    wang.buy(new Demand, 20000, 365);        // 活期2万放一年
    wang.buy(new Fixed(0.021, 365), 50000, 365); // 一年定期5万
    wang.buy(new Fixed(0.021, 365), 30000, 200); // 3万定期提前支取
    wang.buy(new LargeCD(0.031, 365), 200000, 365); // 大额存单20万
    wang.buy(new Fund, 10000, 365);          // 基金1万
    cout << "王同学一年总收益:" << wang.yearProfit()
         << "元" << endl;
    return 0;
}

```

5. 塔防游戏

```
int myrand(){ // 随机生成敌人用的简易随机数
    static int seed = 3;
    seed = (seed * 7 + 11) % 10;
    return seed;
}
class Enemy { // 抽象敌人
public:
    int hp, speed, reward, pos; // 生命、速度、赏金、位置
    Enemy(int h, int s, int r):hp(h),speed(s),reward(r),pos(0){}
    bool dead(){ return hp <= 0; }
    virtual void move(){ pos += speed; } // 每回合前进
};
class Soldier : public Enemy {
public: Soldier():Enemy(30, 1, 5){} // 士兵
};
class Cavalry : public Enemy {
public: Cavalry():Enemy(50, 2, 10){} // 骑兵
};
class Flyer : public Enemy {
public:
    Flyer():Enemy(40, 3, 15){} // 飞行兵, 可重写move走捷径
};
class Boss : public Enemy {
public: Boss():Enemy(300, 1, 100){} // Boss
};
class Tower { // 抽象防御塔
protected:
    int atk, range, pos, level;
public:
    Tower(int a, int r, int p):atk(a),range(r),pos(p),level(1){}
    virtual int cost() = 0; // 造价
    void upgrade(){ level++; atk += atk / 2; } // 花钱升级增强攻击
    void attack(vector<Enemy*>& es) { // 打射程内最前面的敌人
        Enemy* target = NULL;
        for (int i = 0; i < es.size(); i++)
            if (!es[i]->dead()
                && es[i]->pos >= pos - range
                && es[i]->pos <= pos + range)
                if (!target || es[i]->pos > target->pos)
                    target = es[i];
        if (target) target->hp -= atk;
    }
};
class ArrowTower : public Tower {
public:
    ArrowTower(int p):Tower(10, 8, p){}
    int cost(){ return 50; }
};
class CannonTower : public Tower {
public:
    CannonTower(int p):Tower(25, 5, p){}
    int cost(){ return 100; }
};
class MagicTower : public Tower {
public:
```

```

    MagicTower(int p):Tower(18, 10, p){}
    int cost(){ return 120; }
};
class Wave {                // 一波敌人
public:
    static vector<Enemy*> make(int no) {    // 波次越靠后敌人越强
        vector<Enemy*> es;
        for (int i = 0; i < 3 + no; i++) { // 数量随波次增加
            int r = myrand();
            if (r < 5) es.push_back(new Soldier);
            else if (r < 8) es.push_back(new Cavalry);
            else es.push_back(new Flyer);
        }
        if (no == 3) es.push_back(new Boss);    // 最后一波出Boss
        return es;
    }
};
class Game {
    vector<Tower*> towers;
    int gold, life, endPos;
public:
    Game():gold(200),life(10),endPos(20){}
    void build(Tower* t) {        // 金币够才能建塔
        if (gold >= t->cost()){ gold -= t->cost(); towers.push_back(t); }
    }
    void playWave(int no) {        // 打一波：回合制
        vector<Enemy*> es = Wave::make(no);
        while (true) {
            int alive = 0;
            for (int i = 0; i < towers.size(); i++) // 塔先攻击
                towers[i]->attack(es);
            for (int i = 0; i < es.size(); i++) {    // 敌人移动/结算
                if (es[i]->dead()) {
                    if (es[i]->reward) {            // 击杀奖励只发一次
                        gold += es[i]->reward;
                        es[i]->reward = 0;
                    }
                    continue;
                }
                if (es[i]->pos >= endPos) continue; // 已到终点的不重复扣
                es[i]->move();
                if (es[i]->pos >= endPos) life--;    // 到达终点扣命
                else alive++;
            }
            if (alive == 0 || life <= 0) break;
        }
        cout << "第" << no << "波结束, 剩余生命" << life
              << ", 金币" << gold << endl;
    }
    void run() {
        for (int no = 1; no <= 3 && life > 0; no++)
            playWave(no);
        cout << (life > 0 ? "守住了, 胜利!" : "游戏失败") << endl;
    }
};
int main(){
    Game g;

```

```
g.build(new ArrowTower(5)); // 在路线不同位置建三座塔
g.build(new CannonTower(10));
g.build(new ArrowTower(15)); // 金币不够则建塔失败，略
g.run();
return 0;
}
```